EPISODE 13 · FREE COMPANION GUIDE

Build Your Own Free Meta Muse

A 3-Agent AI Content Team With CrewAI + Ollama — $0, No API Key, Runs on Your Laptop

Saba Automations
Social Media Workflows Series — Phase 4: Multi-Agent Systems

WHY THIS GUIDE

The Trend Behind the Build

This week, Meta launched **Muse** — a personal AI agent that browses the web, drafts your emails, manages your calendar, and can even make purchases on your behalf. Grok has its own version ("Bot"), Apple is rebuilding Siri around the same idea, and OpenAI just hired the creator of OpenClaw to build its answer.

Here's the part most headlines skip: one piece of software isn't reliably browsing, emailing, and spending money all by itself. Under the hood, these systems are almost certainly running **multiple specialized agents** that hand work off to each other — one agent browses, another checks your calendar, another confirms the purchase.

> *That's exactly the architecture you're about to build in this guide — for free, running locally, with three agents doing one job each: **Writer** → **Editor** → **Publisher.***

WHAT YOU'LL WALK AWAY WITH

This Guide Covers

- The exact free tools to install (2 tools, 2 lines in your terminal)
- Full working code for a 3-agent content team (Writer, Editor, Publisher)
- The honest limits of running this for free — and when to scale it
- A quick-reference checklist you can follow with zero rewatching

STEP 1

The Two Free Tools

No credit card. No API key. No usage bill. This is the entire toolkit.

Tool	What it does	Cost
Ollama	Runs AI models locally on your own machine — no internet dependency once downloaded	Free forever
CrewAI	Open-source framework purpose-built for coordinating multiple AI agents as a team	Free / open source

Install in your terminal:

```
ollama pull qwen2.5:7b
pip install crewai crewai-tools
```

> ***Minimum specs:*** *~16GB of RAM. That's it — this runs on a normal laptop.*

STEP 2

Define Your 3 Agents

This is the difference between a chatbot and an agent: a chatbot answers, an agent has a **job**. Each agent below gets a role, a goal, and a backstory — that's what gives it a consistent “personality” instead of generic output.

```
from crewai import Agent, Task, Crew
from crewai import LLM

local_llm = LLM(model="ollama/qwen2.5:7b",
                base_url="http://localhost:11434")

writer = Agent(
    role="Content Writer",
    goal="Write an engaging, platform-ready social post",
    backstory="A punchy social copywriter who writes hooks
               that stop the scroll",
    llm=local_llm
)

editor = Agent(
    role="Content Editor",
    goal="Review for clarity, tone, and length — reject
         weak hooks",
    backstory="A ruthless editor with zero tolerance for
               boring openers",
    llm=local_llm
)

publisher = Agent(
    role="Publishing Formatter",
    goal="Format the approved post for the target platform",
    backstory="Knows exact character limits and hashtag
               conventions per platform",
    llm=local_llm
)
```

STEP 3

Wire Up the Handoffs & Run It

The context=[write_task] line below is the important part — that's the editor literally reading the writer's draft before doing anything. Same as a real content team's Slack thread, except instant.

```
write_task = Task(
    description="Write a 100-word Instagram caption
                about {topic}",
    expected_output="A caption with a strong hook
                    in the first line",
    agent=writer
)

edit_task = Task(
    description="Review the caption. If the hook is
                weak, rewrite it stronger.",
    expected_output="A final, approved caption",
    agent=editor,
    context=[write_task]
)

publish_task = Task(
    description="Format the approved caption for
                Instagram with 5 relevant hashtags",
    expected_output="Platform-ready post text",
    agent=publisher,
    context=[edit_task]
)

crew = Crew(
    agents=[writer, editor, publisher],
    tasks=[write_task, edit_task, publish_task],
    verbose=True
)

result = crew.kickoff(inputs={
    "topic": "why most AI agents fail without teamwork"
})
print(result)
```

THE HONEST PART

What This Doesn't Do (Yet)

This runs on your own machine, so it's only as fast as your machine. Fine for generating one post at a time. If you need 50 posts a day for client work, you'll want either a stronger local machine or a paid API for speed. That's the same tradeoff every free tier in this series has had — great for learning and small use, and the exact point where a business needs someone to properly architect and scale it.

QUICK-REFERENCE

Build Checklist

[ ] Install Ollama and pull qwen2.5:7b (or another local model)

[ ] pip install crewai crewai-tools

[ ] Define 3 agents: Writer, Editor, Publisher (role + goal + backstory)

[ ] Define 3 tasks with context=[...] handoffs between them

[ ] Run crew.kickoff() with your topic as input

[ ] Read the verbose output — watch each agent's handoff happen live

[ ] Note your real run time — that's your baseline for scaling later

> ***Next in the series (Episode 14):*** *LangGraph — a workflow that checks its own output and self-corrects before anything gets published. If Episode 13 was "a team that talks to each other," Episode 14 is "a team that catches its own mistakes."*

Made this? Drop a comment on the video with what content task YOU handed to your 3-agent team.